

# **Del software de sistemas al análisis léxico**

## **Repaso de las notas 02 y 03 y trabajo con el Cuadernillo 02**

Manuel Soto Romero    Araceli Liliana Reyes Cabello

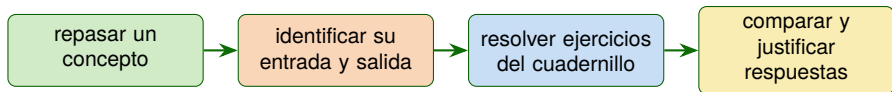
Colegio de Ciencia y Tecnología, UACM

Semestre 2026-2

# Iniciamos con Kahoot

Iniciamos con el Kahoot

# Ruta de trabajo



## Durante la sesión

Alternaremos el repaso de las notas 02 y 03 con el trabajo del **Cuadernillo de ejercicios 02.**

# Abrimos la caja del compilador



## La descripción externa no basta

En su interior, el compilador reconoce unidades, descubre estructuras, comprueba restricciones y transforma representaciones antes de producir su salida.

**El programa cambia de representación, pero debe conservar su significado.**

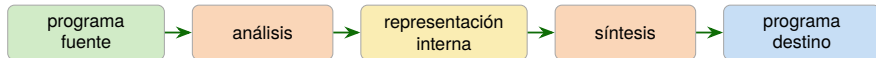
# Dos grandes etapas: análisis y síntesis

## Análisis

Examina el programa fuente para reconocer sus componentes, su estructura y las restricciones que debe satisfacer.

## Síntesis

Toma la representación obtenida por el análisis y construye una representación equivalente en el lenguaje destino.



# Las fases se comunican mediante contratos

| Fase                 | Entrada                              | Salida principal            |
|----------------------|--------------------------------------|-----------------------------|
| Análisis léxico      | Caracteres                           | Secuencia de tokens         |
| Análisis sintáctico  | Tokens                               | Representación estructurada |
| Análisis semántico   | Estructura e información necesaria   | Representación comprobada   |
| Generación de IR     | Representación comprobada            | Representación intermedia   |
| Optimización         | Representación intermedia            | Representación equivalente  |
| Generación de código | Representación validada o intermedia | Programa destino            |

## Pregunta para cada fase

¿Qué recibe?, ¿qué tarea realiza?, ¿qué produce? y ¿quién utiliza el resultado?

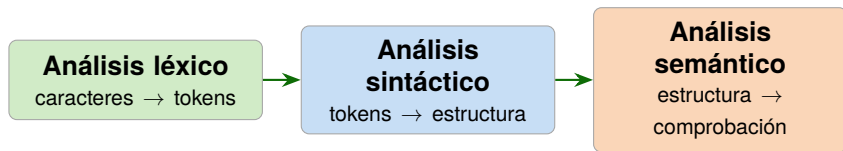
# Trabajo con el cuadernillo

## Cuadernillo de ejercicios 02

### Ejercicios 1 y 2

- ▶ **Esencial — Ejercicio 1:** reconocer los contratos entre fases.
- ▶ **Extensión — Ejercicio 2:** distinguir las tareas de análisis y de síntesis.

# Tres fases para comprender el programa fuente



- ▶ El léxico reconoce unidades elementales.
- ▶ El sintáctico organiza esas unidades según la gramática.
- ▶ El semántico comprueba restricciones como declaraciones y compatibilidad de tipos.



# Cada fase detecta problemas diferentes

| Fase       | Situación que le corresponde                                                |
|------------|-----------------------------------------------------------------------------|
| Léxica     | Ningún patrón permite reconocer el inicio de los caracteres restantes.      |
| Sintáctica | Los tokens existen, pero no pueden organizarse de acuerdo con la gramática. |
| Semántica  | La estructura es válida, pero incumple una regla de declaración o de tipos. |

## Límite de responsabilidad

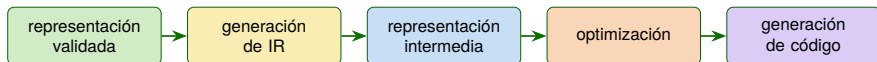
Reconocer un texto como identificador no permite decidir todavía si fue declarado ni qué tipo tiene.

## Cuadernillo de ejercicios 02

### Ejercicios 3 y 4

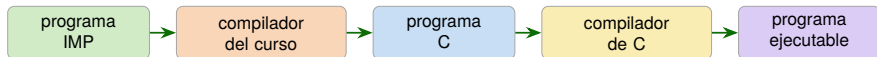
- ▶ **Esencial — Ejercicio 3:** decidir qué fase debe responder en cada situación.
- ▶ **Extensión — Ejercicio 4:** revisar los límites de responsabilidad entre fases.

# De la representación validada al código destino



- ▶ Una IR facilita la comunicación y la transformación del programa.
- ▶ La optimización busca una mejora sin cambiar el comportamiento definido.
- ▶ No todos los compiladores usan exactamente una IR o el mismo número de optimizaciones.

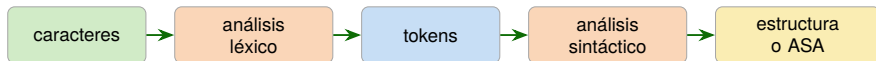
# El compilador del curso y la cadena posterior



## Destino de nuestro compilador

El programa C completa la traducción prevista para el curso. Todavía necesita un compilador de C para continuar la cadena hacia un ejecutable.

# Integrar significa acordar interfaces



## Contrato

La fase consumidora debe conocer la forma y el significado de los datos que recibe.

## Cambio

Si cambia una representación, deben revisarse las fases que la producen y la consumen.

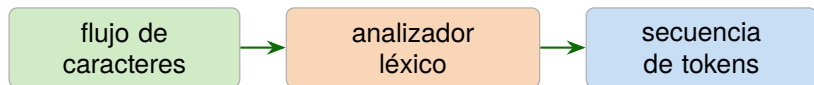
ASA significa *árbol de sintaxis abstracta*; la denominación proviene del inglés *Abstract Syntax Tree*, abreviado AST.

## Cuadernillo de ejercicios 02

### Ejercicios 5 y 6

- ▶ **Esencial — Ejercicio 5:** completar el recorrido de IMP a C.
- ▶ **Extensión — Ejercicio 6:** analizar las interfaces y el efecto de sus cambios.

# Función del analizador léxico



- ▶ Agrupa caracteres en lexemas y produce tokens.
- ▶ Puede omitir espacios o comentarios cuando el lenguaje indica que no son significativos.
- ▶ Conserva atributos necesarios para las fases posteriores.
- ▶ No determina por sí solo la estructura completa, las declaraciones o los tipos.

# Del flujo de caracteres a los tokens

## Fragmento ilustrativo: no define la sintaxis de IMP

```
valor = valor + inc;
```

| Lexema | Clasificación | Resultado descriptivo |
|--------|---------------|-----------------------|
| valor  | identificador | ID(valor)             |
| =      | asignación    | ASIG                  |
| valor  | identificador | ID(valor)             |
| +      | suma          | SUMA                  |
| inc    | identificador | ID(inc)               |
| ;      | terminador    | PYC                   |



# Token, lexema y patrón

## Patrón

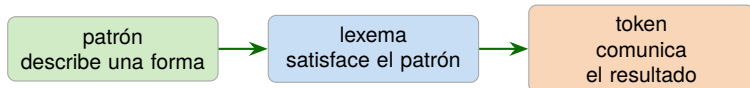
Describe la forma que pueden tener los lexemas asociados con un nombre de token.

## Lexema

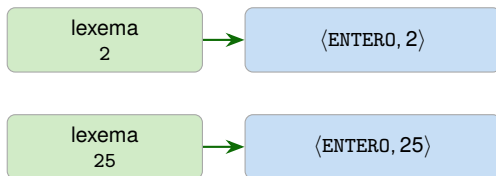
Es la secuencia concreta de caracteres reconocida en el programa fuente.

## Token

Comunica un nombre de token y un valor de atributo opcional.



# ¿Para qué sirve un atributo?



## Mismo nombre, distinta aparición

El nombre ENTERO comunica la categoría; el atributo distingue el valor concreto. De manera semejante, un token ID puede conservar el texto del identificador o una referencia a información asociada.

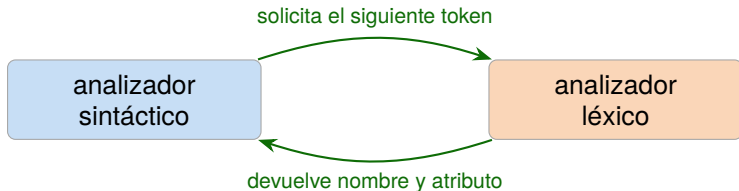
Algunos operadores y signos de puntuación no necesitan atributo: su nombre ya comunica la información requerida.

## Cuadernillo de ejercicios 02

### Ejercicios 7 y 8

- ▶ **Esencial — Ejercicio 7:** relacionar token, lexema, patrón y atributo.
- ▶ **Esencial — Ejercicio 8:** analizar el fragmento ilustrativo `total = 25;`.

# El analizador sintáctico solicita tokens



## Una interfaz frecuente

El analizador léxico consume sólo los caracteres necesarios para reconocer el siguiente lexema y devolver su token.

**La salida léxica debe tener la forma que espera el analizador sintáctico.**

# Qué especificamos y qué apoya Lark

## El equipo especifica

- ▶ qué categorías reconoce el lenguaje;
- ▶ qué forma tiene cada categoría;
- ▶ qué información debe conservar cada token.

## Lark apoya

- ▶ la lectura de la entrada;
- ▶ el reconocimiento según las reglas dadas;
- ▶ la producción de tokens para observar o analizar.

```
parser=None    lexer="basic"    lex()
```

Esta configuración permite observar el analizador léxico de manera independiente.

# Confusiones que debemos evitar

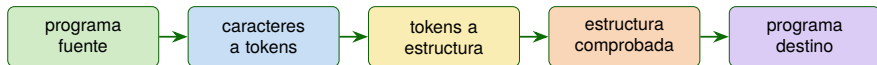
| Confusión                                     | Corrección                                                          |
|-----------------------------------------------|---------------------------------------------------------------------|
| Lexema y patrón son lo mismo.                 | El lexema es concreto; el patrón describe una forma.                |
| El léxico recibe tokens y produce caracteres. | Recibe caracteres y produce tokens.                                 |
| Dos tokens ID tienen el mismo lexema.         | Pueden compartir nombre y tener lexemas y atributos distintos.      |
| La optimización repara la sintaxis.           | Transforma una representación válida sin cambiar su comportamiento. |
| El C generado ya es ejecutable.               | Debe continuar por un compilador de C.                              |
| Lark decide las categorías del lenguaje.      | Lark aplica las categorías y reglas especificadas por el equipo.    |

## Cuadernillo de ejercicios 02

### Ejercicios 9 y 10

- ▶ **Esencial — Ejercicio 9:** completar la comunicación para solicitar el siguiente token.
- ▶ **Extensión — Ejercicio 10:** detectar y corregir confusiones del repaso completo.

# Conclusión



## Idea de cierre

El compilador organiza transformaciones mediante fases con responsabilidades e interfaces claras. El análisis léxico inicia el recorrido interno: reconoce lexemas, produce tokens y comunica la información que necesita la fase sintáctica.

**Comprender cada contrato permite construir y probar el compilador de manera acumulativa.**



# Referencias

1. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Nota de clase 02: Estructura y fases de un compilador*. Colegio de Ciencia y Tecnología, UACM.
2. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Nota de clase 03: Fundamentos del análisis léxico*. Colegio de Ciencia y Tecnología, UACM.
3. Soto Romero, M. y Reyes Cabello, A. L. (2026). *Programación de Sistemas. Cuadernillo de ejercicios 02: Del software de sistemas al análisis léxico*. Colegio de Ciencia y Tecnología, UACM.
4. Aho, A. V., Lam, M. S., Sethi, R., y Ullman, J. D. (2008). *Compiladores: principios, técnicas y herramientas* (2.<sup>a</sup> ed.). Pearson Educación. ISBN 978-970-26-1133-2.
5. Thain, D. (2026). *Introduction to Compilers and Language Design* (2.<sup>a</sup> ed.). University of Notre Dame.
6. Vilar Torres, J. M. (2010). *Estructura de los compiladores e intérpretes*. Universitat Jaume I.
7. Vilar Torres, J. M. (2010). *Analizador léxico*. Universitat Jaume I.